

fkcompute, an efficient F_K invariant calculator

Paul Orland, Davide Passaro, Lara San Martín Suárez, Toby Saunders-A’Court, Josef Svoboda

February 2026

Contents

1	Introduction	1
2	The fkcompute package	2
3	Mathematical background	3
3.1	The F_K and F_L invariants	3
3.2	Large-color R -matrices	4
3.3	State sums	5
4	Algorithm	6
4.1	Topological Preprocessing and State Coordinates	6
4.2	Phase 1: Constructing an Acceptable Inversion Datum	7
4.3	Phase 2: Linear Constraint System and ILP Export	7
4.4	Phase 3: Integer-Point Enumeration and R -Matrix Evaluation	8
5	Usage and examples	9
5.1	Braid input conventions	9
5.2	Command-line interface	10
5.3	Output format	11
5.4	Python API	11
5.5	Mathematica interface	12
5.6	Worked examples	12
6	Performance	12
7	Validation and dataset	12
7.1	Regression against reference tables . .	13
7.2	The $q \rightarrow 1$ (Alexander/MMR) specialization	13
7.3	The first q -derivative at $q = 1$ (first MMR correction)	14

7.4	Dataset availability	14
-----	--------------------------------	----

8	Limitations and future work	15
----------	------------------------------------	-----------

1 Introduction

Modern topology increasingly relies on the computation of invariants to distinguish, classify, and analyze geometric and algebraic structures. For knots, links, and three-manifolds, these invariants are used not only to distinguish examples, but also to test conjectures, build tables, and guide new theoretical work.

This computational role has grown as the invariants themselves have become more refined. Leading invariants have grown in sophistication, with their computation has correspondingly becoming more intricate and resource-intensive. Classical invariants such as the Alexander polynomial and the Jones polynomial are relatively inexpensive to compute. By contrast, modern invariants such as Khovanov homology, knot Floer homology, colored Jones polynomials, and the Gukov–Manolescu series involve larger algebraic structures, recursive formulas, or state sums whose cost grows quickly with the diagrammatic complexity of the knot involved.

At the same time, large datasets of invariants have become increasingly important. Traditional mathematical investigation is now often complemented by data-driven methods, including techniques from data science and machine learning, which are beginning to play a substantive role in contemporary research. This shift makes reproducibility and accessibility more urgent. As the number of invariants grows, and as their computation depends on increasingly specialized algorithms, it becomes harder to dis-

tribute implementations, verify results, and collaborate on further development. The problem is especially pronounced when the relevant software consists of complex, domain-specific code.

This trend has raised the demand for higher accuracy in computation and more comprehensive datasets. If invariants are to be used for experimentation, classification, and comparison, then their values must be computed reliably and made available in a form that other researchers can inspect, reproduce, and extend. Datasets are valuable, but a dataset without the code that produced it is difficult to audit. For newer invariants, this point is especially important: different normalizations, expansion conventions, and truncation choices can lead to superficially different outputs.

In this note, we present **fkcompute**, a Python package which aims to fill the current gap for *open* computational tools for the F_K invariant. The F_K invariant emerged from the study of the $\widehat{Z}$ invariant, a q -series quantum invariant of closed 3-manifolds [?, ?]. From this perspective, the F_K invariant can be interpreted as a $\widehat{Z}$ invariant for knot and link complements. More broadly, the $\widehat{Z}$ invariant itself was introduced as part of a program aimed at developing categorified invariants of 3-manifolds, analogous in spirit to how Khovanov homology categorifies the Jones polynomial for knots, and F_K plays a central role in this framework.

The **fkcompute** package provides a suite of utilities and algorithms that implement the necessary algebraic and combinatorial procedures to construct the invariant directly from braid data. The implementation supports homogeneous braids and, more generally, the class of *nice* knots and links as defined by Park [?], which conjecturally includes all fibered knots; thereby covering a broad and practically relevant set of examples. We tested the software on *nice* knots with up to 12 crossings and *nice* links up to 10 crossings and 3 components, demonstrating both robustness and computational feasibility within this range.

2 The **fkcompute** package

fkcompute is provided as a Python package, as a standalone command-line interface, and as a Mathematica paclet allowing users to compute the F_K invariant directly from a braid presentation in a flexible and accessible manner. The package is designed with extensibility as a central goal: its source code is publicly available and structured to facilitate modification, extension, and integration into future computational workflows or research projects.

Users may interact with **fkcompute** programmatically by importing it into Python scripts, enabling automated computation, batch processing, and integration with existing mathematical software pipelines. Alternatively, **fkcompute** can be used as a command-line tool, providing a straightforward interface for direct computation from the terminal without requiring additional programming. For users working in Mathematica, we also provide a Wolfram Language Paclet (**FkCompute**) which calls the Python package, allowing computations and symbolic expressions to be used directly in notebooks.

fkcompute is freely available for download through its public GitHub repository, providing open access to both the source code and accompanying documentation. Precomputed F_K tables used for validation and benchmarking are distributed separately from the core source repository.

Internally, **fkcompute** leverages high-performance external libraries to ensure computational efficiency and reliability: polynomial arithmetic is handled using the FLINT library, a C++ library optimized for fast and precise computation with polynomials and power series, while Gurobi is used for constraint validation, enabling robust and efficient resolution of the linear and combinatorial conditions that arise in the construction of the invariant.

The package does not claim to define the F_K series for arbitrary knots. It implements the inverted state-sum construction whenever an acceptable inversion datum is found. In particular, it covers homogeneous braids and many non-homogeneous examples. Each calculation takes as input a braid word, a truncation degree for the x -variables and optionally a prescribed inversion datum. After completing the computation

it returns a truncated $F_K(x; q)$ series for knots or a $F_L(x_1, \dots, x_C; q)$ series for links as well as the inversion datum used and other metadata regarding the computation.

3 Mathematical background

In this section we recall some of the mathematical background of the F_K invariant, as well as some of its properties which serve as checks on the computations performed by `fkcompute`. In this section we will focus on the case of F_K for knots, although similar considerations for links are also leveraged to assess the computations of `fkcompute`.

3.1 The F_K and F_L invariants

The F_K invariant arises from the same program which produced the $\widehat{Z}$ invariants of closed 3-manifolds. $\widehat{Z}$ invariants are q -series quantum invariants, introduced from considerations in supersymmetric quantum field theory in 6D. $\widehat{Z}$ invariants are expected to recover the Witten–Reshetikhin–Turaev invariants through radial limits at roots of unity and to admit homological refinements and to admit a categorification [?, ?]. Gukov and Manolescu proposed an analogue of this construction for knot complements [?]. The F_K invariant will be a $C + 1$ series, where C is the number of components of the link it is describing. For a knot $K \subset S^3$ with a single component, this invariant is the two-variable series $F_K(x; q)$ whereas for a link $L \subset S^3$ with C components the invariant $F_L(x_1, \dots, x_C; q)$ is a function of $C + 1$ variables.

In this sense, F_K should be regarded as the knot-complement counterpart of $\widehat{Z}$. This interpretation is particularly natural from the perspective of surgery. Since closed 3-manifolds may be obtained by Dehn surgery on knots and links in S^3 , one expects the $\widehat{Z}$ invariants of the resulting closed manifolds to be recoverable from the corresponding knot-complement series. More precisely, if $S_{p/r}^3(K)$ denotes the result of p/r surgery on K , the expected surgery relation

has the form

$$\widehat{Z}_b \left(S_{p/r}^3(K) \right) = \varepsilon q^\delta \mathcal{L}_{p/r}^{(b)} \left[\left(x^{\frac{1}{2r}} - x^{-\frac{1}{2r}} \right) F_K(x, q) \right]. \quad (1)$$

Here $\mathcal{L}_{p/r}^{(b)}$ denotes the Laplace transform determined by the surgery coefficient and by the label b , which corresponds to a Spin^c structure on the surgered manifold. Formula (1) is one of the principal motivations for the explicit computation of F_K : it relates a two-variable invariant of a knot complement to one-variable q -series invariants of closed 3-manifolds.

The F_K series is also constrained by its relationship with the colored Jones polynomial. Let $J_{K,n}(q)$ denote the reduced n -colored Jones polynomial, normalized so that the unknot has value one. The Melvin–Morton–Rozansky expansion studies $J_{K,n}(q)$ in the large-color regime

$$q = e^\hbar, \quad x = q^n = e^{n\hbar}, \quad (2)$$

where x is held fixed and $\hbar \rightarrow 0$. In this limit, one obtains an asymptotic expansion of the form

$$J_{K,n}(e^\hbar) \sim \sum_{j \geq 0} \frac{P_{K,j}(x)}{\Delta_K(x)^{2j+1}} \hbar^j, \quad (3)$$

where $\Delta_K(x)$ is the Alexander polynomial of K . The leading term in (3) is the classical Melvin–Morton term and is given by the inverse of the Alexander polynomial.

Gukov and Manolescu conjectured that F_K is obtained by reorganizing this perturbative expansion into an integral two-variable q -series. In order to state this relation, it is convenient to use the normalized series

$$f_K(x, q) = \frac{F_K(x, q)}{x^{1/2} - x^{-1/2}}. \quad (4)$$

The expansion of $f_K(x, e^\hbar)$ at $\hbar = 0$ is expected to recover the Melvin–Morton–Rozansky expansion, with the rational functions in x expanded at $x = 0$ and $x = \infty$ rather than at $x = 1$. Consequently,

$$\lim_{q \rightarrow 1} f_K(x, q) = \text{s.e.} \left(\frac{1}{\Delta_K(x)} \right), \quad (5)$$

where s. e. denotes the symmetric expansion, namely the average of the Laurent expansions at $x = 0$ and at $x = \infty$. Equivalently,

$$\lim_{q \rightarrow 1} F_K(x, q) = \text{s. e.} \left(\frac{x^{1/2} - x^{-1/2}}{\Delta_K(x)} \right). \quad (6)$$

The first derivative in the q -direction provides a further constraint. The first q -derivative of the normalized series at $q = 1$ is expected to recover the first nontrivial term in the Melvin–Morton–Rozansky expansion:

$$D_q f_K(x, q)|_{q=1} = \text{s. e.} \left(\frac{P_{K,1}(x)}{\Delta_K(x)^3} \right), \quad (7)$$

following from Park’s formulation of Gukov and Manolescu’s conjecture

$$\frac{F_K(x, q = e^{\hbar})}{x^{1/2} - x^{-1/2}} = \sum_{j \geq 0} \frac{P_K(j; x)}{\Delta_K(x)^{2j+1}} \frac{\hbar^j}{j!}. \quad (8)$$

Thus the specialization at $q = 1$ tests the Alexander-polynomial term of the expansion, while the first q -derivative tests the next Melvin–Morton–Rozansky coefficient. These identities provide the basic consistency checks used below for the computations implemented in `fkcompute`.

3.2 Large-color R -matrices

The computation of F_K used in `fkcompute` follows Park’s large-color R -matrix construction [?, ?]. It is formally analogous to the Reshetikhin–Turaev construction of the colored Jones polynomial: a braid is decorated by representation-theoretic data, each crossing contributes a local R -matrix weight, and the braid is closed by taking a reduced quantum trace.

We first fix the notation used throughout Sections 2.1 and 2.2. Let β be a braid on N strands whose closure is the knot or link under consideration. Let V_β denote the set of crossings of β , and let E_β denote the set of oriented segments obtained by cutting the braid at its crossings and then identifying the top and bottom endpoints according to the braid closure. For a crossing $c \in V_\beta$, write

$$e_{\text{BL}}(c), \quad e_{\text{BR}}(c), \quad e_{\text{TL}}(c), \quad e_{\text{TR}}(c)$$

for the adjacent bottom-left, bottom-right, top-left, and top-right segments. A state is a function

$$s : E_\beta \rightarrow \mathbb{Z}.$$

At a crossing c , we abbreviate

$$\begin{aligned} i &= s(e_{\text{BL}}(c)), & j &= s(e_{\text{BR}}(c)), \\ i' &= s(e_{\text{TL}}(c)), & j' &= s(e_{\text{TR}}(c)). \end{aligned}$$

The local state variables are required to satisfy the conservation law

$$i + j = i' + j', \quad (9)$$

or equivalently

$$s(e_{\text{BL}}(c)) + s(e_{\text{BR}}(c)) = s(e_{\text{TL}}(c)) + s(e_{\text{TR}}(c)).$$

This conservation law is enforced in the formulas below by the Kronecker delta $\delta_{i+j, i'+j'}$.

Suppose first that c is a positive crossing. Let x be the variable assigned to the strand running from the bottom-left segment to the top-right segment, and let y be the variable assigned to the strand running from the bottom-right segment to the top-left segment. With these conventions, the extended large-color R -matrix is

$$\begin{aligned} \check{R}(x, y)_{i,j}^{i',j'} &= \delta_{i+j, i'+j'} q^{\frac{j+j'+1}{2}} x^{-\frac{i'+j+1}{4}} y^{-\frac{3j'-i+1}{4}} \\ &\times q^{jj'} \begin{bmatrix} i \\ i-j' \end{bmatrix}_{q^{-1}} \prod_{1 \leq r \leq i-j'} (1 - q^{j+r} y^{-1}), \quad (10) \end{aligned}$$

if $i \geq j' \geq 0$ or $0 > i \geq j'$,

$$\begin{aligned} \check{R}(x, y)_{i,j}^{i',j'} &= \delta_{i+j, i'+j'} q^{\frac{j+j'+1}{2}} x^{-\frac{i'+j+1}{4}} y^{-\frac{3j'-i+1}{4}} \\ &\times q^{jj'} \begin{bmatrix} i \\ j' \end{bmatrix}_{q^{-1}} \prod_{0 \leq r \leq j'-i-1} (1 - q^{j-r} y^{-1}), \quad (11) \end{aligned}$$

if $j' \geq 0 > i$, and $\check{R}(x, y)_{i,j}^{i',j'} = 0$ otherwise.

For a negative crossing one uses the inverse large-color R -matrix:

$$\begin{aligned} \check{R}^{-1}(x, y)_{i,j}^{i',j'} &= \delta_{i+j, i'+j'} q^{-\frac{i+i'+1}{2}} x^{\frac{3i'-j+1}{4}} y^{\frac{j'+i+1}{4}} \\ &\times q^{-ii'} \begin{bmatrix} j \\ j-i' \end{bmatrix}_{q^{-1}} \prod_{1 \leq r \leq j-i'} (1 - q^{-i-r} x), \quad (12) \end{aligned}$$

if $j \geq i' \geq 0$ or $0 > j \geq i'$,

$$\check{R}^{-1}(x, y)_{i, j}^{i', j'} = \delta_{i+j, i'+j'} q^{-\frac{i+i'+1}{2}} x^{\frac{3i'-j+1}{4}} y^{\frac{j'+i+1}{4}} \times q^{-ii'} \left[\begin{matrix} j \\ i' \end{matrix} \right]_{q^{-1}} \prod_{0 \leq r \leq i'-j-1} (1 - q^{-i+r} x)^{-1}, \quad (13)$$

if $i' \geq 0 > j$, and

$$\check{R}^{-1}(x, y)_{i, j}^{i', j'} = 0$$

otherwise.

Here

$$(a; q)_m = \prod_{r=0}^{m-1} (1 - aq^r), \quad \left[\begin{matrix} n \\ k \end{matrix} \right]_q = \frac{(q^n; q^{-1})_k}{(q; q)_k} \quad (14)$$

for $k \geq 0$, with the convention that an empty product is equal to one.

For a link, the variables x and y in the local formula are the variables assigned to the two link components passing through the crossing. For a knot, there is only one variable, so $x = y$.

3.3 State sums

The large-color R -matrices from Section 2.1 give local weights. To obtain a well-defined series, one must also specify which integer values are allowed on each segment of the braid. This choice is encoded by an *inversion datum*.

An inversion datum is a map

$$\iota : E_\beta \longrightarrow \{+, -\}.$$

We use the convention that a segment marked $+$ is assigned a non-negative integer and a segment marked $-$ is assigned a negative integer:

$$\iota(e) = + \implies s(e) \geq 0, \quad \iota(e) = - \implies s(e) < 0.$$

The sign assignment must be locally compatible with the non-zero cases of the extended R -matrices. In the notation

$$\begin{matrix} \iota(e_{\text{TL}}) & \iota(e_{\text{TR}}) \\ \iota(e_{\text{BL}}) & \iota(e_{\text{BR}}) \end{matrix},$$

the allowed local patterns are

$$\begin{matrix} - & + & & + & - & & - & - & & - & + & & + & + \\ + & - & , & - & + & , & - & - & , & - & + & , & + & + \end{matrix} \quad (15)$$

for a positive crossing, and

$$\begin{matrix} - & + & & + & - & & - & - & & + & - & & + & + \\ + & - & , & - & + & , & - & - & , & + & - & , & + & + \end{matrix} \quad (16)$$

for a negative crossing. Thus, at every crossing, the number of incoming $-$ -marked segments equals the number of outgoing $-$ -marked segments.

The role of the inversion datum is partly formal and partly computational. Formally, it specifies which segments are treated by the highest-weight and lowest-weight large-color R -matrices. The geometric meaning of this choice is not yet fully understood. What is essential for the present paper is that an inversion datum gives a concrete finite-order procedure for computing the corresponding state sum.

We will call an inversion datum acceptable when the associated state sum is well-defined coefficient by coefficient in the x -variables. More precisely, for each fixed x -degree, only finitely many states should contribute. This condition is what makes the formal sum computable: the coefficient of any fixed monomial in the x -variables can then be obtained by an ordinary finite calculation.

For homogeneous braids, Park showed that such sign data exist and have a simple column-wise description [?]. A braid is homogeneous if, for each generator index k , all occurrences of σ_k in the braid word have the same sign. Following Park, one labels the right-hand segments of each positive crossing by $+$, and the right-hand segments of each negative crossing by $-$. Homogeneity ensures that these labels are consistent along each column of the braid. Thus all columns except the leftmost one are determined. The leftmost column may be labelled entirely by $+$ or entirely by $-$; either choice gives the same construction.

For non-homogeneous braids, there is no such automatic column-wise rule. Instead, one must search for an inversion datum satisfying the local sign conditions above and the coefficientwise finiteness condition. In this sense, the existence of an acceptable

inversion datum is the practical condition that makes the inverted state sum computable. Knots and links admitting such data are often called *nice*.

Let $\Omega(\iota)$ denote the set of states $s : E_\beta \rightarrow \mathbb{Z}$ satisfying the sign bounds determined by ι , the conservation law (9), and the local inequalities required for the corresponding R -matrix entry to be nonzero. For $s \in \Omega(\iota)$, let

$$R_c(s) = \begin{cases} \check{R}(x_a, x_b)_{i,j}^{i',j'}, & c \text{ positive,} \\ \check{R}^{-1}(x_a, x_b)_{i,j}^{i',j'}, & c \text{ negative,} \end{cases}$$

where x_a and x_b are the variables attached to the two strands meeting at c .

We use the reduced trace convention. Let $b_1, \dots, b_N$ be the bottom segments of the braid, identified with the corresponding top segments after closure. The first strand is left open. Its state is fixed to the ground value

$$s_0(b_1) = \begin{cases} 0, & \iota(b_1) = +, \\ -1, & \iota(b_1) = -. \end{cases}$$

We therefore sum over

$$\Omega_0(\iota) = \{s \in \Omega(\iota) : s(b_1) = s_0(b_1)\}.$$

For each closed strand b_r , $r = 2, \dots, N$, the quantum trace contributes

$$\mu_r(s) = x_{\ell(r)}^{-1/2} q^{-1/2-s(b_r)},$$

where $\ell(r)$ is the link component containing b_r . In the knot case, this is simply $x^{-1/2} q^{-1/2-s(b_r)}$.

The contribution of a state $s \in \Omega_0(\iota)$ is

$$P_\iota(s) = \left(\prod_{r=2}^N x_{\ell(r)}^{-1/2} q^{-1/2-s(b_r)} \right) \left(\prod_{c \in V_\beta} R_c(s) \right). \quad (17)$$

The inverted state sum is

$$Z_\iota^{\text{inv}}(\beta) = (-1)^{\mathbf{s}(\iota)} \sum_{s \in \Omega_0(\iota)} P_\iota(s). \quad (18)$$

Here $\mathbf{s}(\iota)$ is the number of closed components of the simple multicycle determined by the $--$ -marked segments; open components, including the one containing the open strand, are not counted.

For a knot, the corresponding F_K series is normalized by

$$F_K(x, q) = (x^{1/2} - x^{-1/2}) Z_\iota^{\text{inv}}(\beta). \quad (19)$$

For a link, the same construction uses one variable for each link component and the corresponding reduced trace convention.

4 Algorithm

The implementation in `fkcompute` evaluates the inverted state sum (18) by a three-phase pipeline. Phase 1 produces an inversion datum ι (called a *sign assignment* in the code) that is compatible with the braid and yields a coefficientwise finite truncation to the requested x -degree. Phase 2 turns the definition of $\Omega_0(\iota)$ into a reduced integer linear constraint system, eliminates dependent state variables, and exports a compact “ILP” description. Phase 3 (a compiled C++ binary) enumerates the integer points in this polyhedron and evaluates the R -matrix weights using FLINT-backed polynomial arithmetic.

4.1 Topological Preprocessing and State Coordinates

Internally, a braid word is represented as a list $\beta = [g_1, \dots, g_n]$ with each $g_t \in \pm\{1, \dots, N-1\}$. The sign of g_t is the crossing sign and $|g_t| = k$ means that, at height t , strands $k-1$ and k cross. The Python class `BraidTopology` realizes the closure of β as a lattice diagram with coordinates (i, t) where $i \in \{0, \dots, N-1\}$ is the strand index and $t \in \{0, \dots, n\}$ is the level between crossings. At each crossing at level t with $|g_t| = k$, the four corners are

$$(k-1, t), (k, t), (k-1, t+1), (k, t+1), \quad (20)$$

which correspond to bottom-left, bottom-right, top-left, and top-right, respectively.

Not every lattice position (i, t) carries an independent state variable: if strand i does not participate in the crossing at height t , then the integer label is transported unchanged through that layer. Accordingly, `BraidTopology` maintains a map $\text{state}(i, t)$

which identifies (i, t) with the canonical state variable representing the segment of the diagram containing that position. Equivalently, the set of state variables is the set of segments obtained by cutting the braid diagram at the crossings, as in Section 2.2. The closure is encoded by the periodicity aliases

$$\text{state}(i, n) \equiv \text{state}(i, 0), \quad i = 0, \dots, N - 1, \quad (21)$$

which correspond to the identifications of top and bottom endpoints under braid closure. The component structure of the closure is computed by traversing the lattice diagram; the resulting component indices are used throughout to decide which topological variable x_c is attached to a given strand segment.

4.2 Phase 1: Constructing an Acceptable Inversion Datum

Phase 1 is implemented via `find_sign_assignment`. The braid is first classified as homogeneous or non-homogeneous.

Homogeneous braids. If β is homogeneous, `fkcompute` constructs the inversion datum deterministically using Park’s column-wise rule: for each generator index k , all occurrences of σ_k have the same sign, and the code assigns a single strand sign ± 1 to that column. This is expanded to a sign assignment on all segments by following the component decomposition of the closure. Given these signs, local compatibility checks are performed every crossing and one of four crossing types $R1, R2, R3, R4$ is assigned, corresponding exactly to the four nonzero cases in (10)–(13).

Non-homogeneous braids (search over multicycle candidates). For non-homogeneous braids, a column-wise rule is unavailable. Instead, `fkcompute` searches for an inversion datum by enumerating *multicycle candidates*. The search is performed over braid variants obtained by cyclic rotation of the braid word and, optionally, a horizontal mirror. These transformations do not change the isotopy type of the closure but can dramatically change the structure of the sign search.

The permutation enumeration begins by labelling arc-segments at the corners of each crossing. Concretely, integer labels are assigned to the bottom-left, bottom-right, top-left, and top-right corners of every crossing by walking the successor map on entry corners; for knots one obtains labels in $\{1, \dots, 2n + 1\}$, while for links additional labels appear, one for each component cycle of the successor map. The routine then records, for each crossing, the pair of arc labels seen by the overstrand and understrand.

From this data, permutations π of the arc labels satisfying a fixed list of local options at each crossing are lazily generated with each permutation inducing a sign diagram by the rule

$$\text{sign}(\ell) = \begin{cases} +1, & \pi(\ell) = \ell, \\ -1, & \pi(\ell) \neq \ell, \end{cases} \quad (22)$$

for each arc label ℓ ; these signs are then grouped by link component to match the inversion datum format used throughout the package.

Candidate sign assignments are tested in parallel using Python `multiprocessing`. For each candidate, the local validation step computes the crossing sign matrix and rejects assignments that do not realize one of the allowed local patterns (15)–(16). If the local test passes, the candidate is subjected to a global boundedness check at the requested degree. This boundedness check is the computational incarnation of the acceptability condition from Section 2.2: after adding the degree truncation inequalities, the resulting integer polyhedron must be nonempty and bounded. In `fkcompute`, this is certified by building the reduced constraint system (Phase 2 machinery) and then calling a Gurobi-based boundedness test, which maximizes each free integer parameter in turn and rejects the candidate if any direction is unbounded.

4.3 Phase 2: Linear Constraint System and ILP Export

Starting from a braid topology and an inversion datum, the code generates the following families of relations.

Boundary conditions. The reduced trace leaves strand 0 open and fixes its endpoints to the ground state. In the code this is the pair of constraints

$$\text{state}(0, 0) = s_0, \quad \text{state}(0, n) = s_0, \quad (23)$$

with $s_0 = 0$ when the open strand is marked $+$ and $s_0 = -1$ when it is marked $-$ (cf. (17)).

Closure (periodicity). For every strand index i , the closure imposes the aliases (21). In the constraint language, these are **Alias** relations.

Sign bounds. The inversion datum specifies whether each segment carries a nonnegative or negative integer. In the code, a sign assignment is expanded to a function $(i, t) \mapsto \pm 1$ on lattice locations. For each state variable u (each segment), this yields one inequality of the form

$$u \geq 0 \quad \text{or} \quad u \leq -1. \quad (24)$$

These are stored as **Leq** constraints against the literals `[0]` and `[-1]`.

Crossing relations. At every crossing, the conservation law (9) is added. In addition, for crossings of type $R1$ or $R4$ the code adds two inequalities ensuring that the q -binomial and q -Pochhammer parameters are nonnegative. In the coordinate convention (20), these are

$$\begin{aligned} R1 : \quad & \text{state}(k, t+1) \leq \text{state}(k-1, t), \\ & \text{state}(k, t) \leq \text{state}(k-1, t+1), \\ R4 : \quad & \text{state}(k-1, t) \leq \text{state}(k, t+1), \\ & \text{state}(k-1, t+1) \leq \text{state}(k, t). \end{aligned} \quad (25)$$

For types $R2$ and $R3$ the corresponding nonnegativity conditions are automatic from the sign bounds (24) (the relevant differences are strictly positive because the two diagonal entries have opposite signs), so no extra inequalities are required.

Reduction and parametrization. The raw relations are simplified by a fixed-point reduction: constants are propagated through aliases, symmetric inequalities ($a \leq b$ and $b \leq a$) are turned into equalities,

unary conservation relations are turned into aliases, and vacuous constraints are removed. From the reduced system, the code assigns fresh symbols to the remaining free variables and propagates these to all state variables using the equalities and “sum-alias” instances of conservation. The outcome is an explicit affine-linear expression

$$\text{state}(i, t) = c_{i,t} + \sum_{r=1}^m M_{i,t,r} a_r, \quad (26)$$

in a small set of free integer parameters $a_1, \dots, a_m$.

Finally, the code rewrites all inequalities and all x -degree truncation conditions in these parameters. Single-variable inequalities are used to normalize the sign of each parameter: if a parameter is constrained by $a_r \leq -1$, the substitution $a_r = -b_r - 1$ is performed so that all parameters become nonnegative. The resulting data—the braid and component metadata, a list of linear inequalities $0 \leq \alpha + \sum \beta_r b_r$, and the full assignment matrix (26) evaluated at every lattice location—is serialized into a compact CSV file. This file is the only input consumed by the C++ backend.

4.4 Phase 3: Integer-Point Enumeration and R -Matrix Evaluation

Phase 3 is performed by the executable `fk_main`. It parses the CSV into an `FKConfiguration` consisting of: the braid word (generator indices), the crossing types 1, 2, 3, 4 corresponding to $R1, R2, R3, R4$, the component indices for each crossing and each closed strand, a list of linear “criteria” rows encoding the x -degree truncation, auxiliary inequalities from Phase 2, and the affine assignment matrix (26) at all lattice locations.

Validated bounded criteria. The C++ code must turn the degree constraints into effective upper bounds for all free parameters in order to enumerate integer points. Because the criteria are stored as general linear forms in the nonnegative parameters, they do not necessarily exhibit an explicit upper bound for each parameter. The routine

`findValidCriteria` therefore searches for an *auxiliary* collection of bounding rows in which every parameter appears with a negative coefficient in some row whose remaining coefficients are nonpositive. If the original degree criteria already have this monotonicity property, they are used directly. Otherwise, `fk_main` performs a breadth-first exploration in the space of rows obtained by repeatedly replacing a criterion row c by $c + \frac{1}{2}u$, where u ranges over the supporting inequalities (the auxiliary inequalities together with the original criteria). Since every supporting inequality is itself enforced during enumeration, replacements of the form $c \mapsto c + \frac{1}{2}u$ only weaken the row constraint and are therefore redundant; they are used solely to derive explicit upper bounds for the parameters.

Enumerating integer points. After a bounded criterion set is found, the admissible parameter vectors are precisely the integer points

$$\mathcal{P} = \left\{ b \in \mathbb{Z}_{\geq 0}^m : \text{all exported inequalities hold} \right\}. \quad (27)$$

The enumerator first derives per-variable bounds from the validated criteria and then performs an iterative depth-first search over variable assignments. At each step, a chosen inequality with a negative coefficient in the next variable provides an explicit upper bound for that variable after substituting the already-fixed values. The first variable is parallelized with OpenMP, and each branch checks the full inequality set before accepting a point. The resulting list of points is exactly the set of admissible integer parameter assignments for which the truncated inverted sum is finite.

Evaluating the R -matrix product. For each point $b \in \mathcal{P}$, the engine evaluates the affine assignment matrix to recover the numerical state values at every lattice location (i, t) ; in particular, at each crossing one obtains the quartet (i, j, i', j') used in Section 2.1. The contribution of b is computed in a factorized form mirroring the local weights (10)–(13): the monomial prefactors in q and the x_c are accumulated as explicit exponent offsets, while the

q -binomial and (inverse) q -Pochhammer factors are expanded as polynomials and multiplied in. Concretely, for each crossing type $r \in \{1, 2, 3, 4\}$, the code multiplies by a product of a q -binomial $\begin{bmatrix} \cdot \\ \cdot \end{bmatrix}_q$ and either $(xq^*; q)_*$ or its inverse. Inverse Pochhammer factors are expanded as geometric series and truncated using the remaining x -degree budget so that no term beyond the requested degree is ever constructed. Polynomial arithmetic is performed using FLINT multivariate polynomials together with thread-safe caches for q -binomials, Pochhammers, and per-crossing products.

Normalization and output. After summing the contributions over all points, the final reduced-trace normalization is applied by multiplying by $(-1 + x_0)$ and truncating in each topological variable. The result is written to JSON together with metadata recording the number of variables and the constant fractional offsets in the q - and x -powers. The Python layer reads this JSON and, if requested, converts it to a symbolic expression by re-introducing the fractional power offsets.

5 Usage and examples

The package is distributed as a Python library, a command-line tool, and a Mathematica paclet. All three interfaces ultimately call the same pipeline from Section 4: Phase 1 (inversion/sign assignment), Phase 2 (ILP export), and Phase 3 (compiled enumeration and R -matrix evaluation). In this section we describe the supported entry points and illustrate typical workflows.

5.1 Braid input conventions

Throughout the package a braid on N strands is encoded as a list of signed generator indices

$$\beta = [g_1, \dots, g_n], \quad g_t \in \pm\{1, \dots, N-1\}.$$

The integer $|g_t| = k$ corresponds to the Artin generator σ_k and the sign of g_t specifies the crossing sign

(positive for σ_k and negative for σ_k^{-1}). The computation is always performed on the *closure* of this braid word.

On the command line, braid words may be passed as a string in any of the following equivalent formats:

```
"[1, -2, 1, -2]"      (JSON-style)
"1,-2,1,-2"           (comma-separated)
"1 -2 1 -2"           (space-separated)
```

In a shell, the braid argument should be quoted so that a negative generator is not misinterpreted as a command-line flag.

5.2 Command-line interface

After installation, the entry point `fk` is available. Running `fk --help` shows the global help as well as the available subcommands. The CLI is intentionally split into a minimal “one-off” mode for quick exploration and a configuration-file mode which exposes the full set of computation parameters.

Interactive wizard. Invoking `fk` with no additional arguments launches an interactive terminal wizard (a richer version is used when the optional dependency `rich` is installed). The wizard prompts for a braid word and degree, offers a preset choice (single-threaded vs. parallel), and then runs the three-phase pipeline while reporting progress.

In addition to the default wizard, one may explicitly run

```
fk interactive
fk interactive --enhanced
fk interactive --quick
```

The `--quick` option skips nonessential prompts and uses sensible defaults. In either mode the wizard can optionally (i) save intermediate files to disk (inversion JSON, ILP CSV, and the final result JSON), and (ii) print a symbolic rendering of the output using SymPy (if installed).

Simple mode (single braid). The subcommand

```
fk simple "[1,1,1]" 2
```

computes the truncated invariant for the closure of the braid and prints the result as JSON. This “simple” mode is a convenience wrapper intended for quick experiments; it uses defaults for parallelism and saving, and exposes only a small number of options.

If SymPy is installed (`pip install "fkcompute[symbolic]"`), one may ask for a human-readable expression instead of raw JSON:

```
fk simple "[1,1,1]" 2 --symbolic
fk simple "[1,1,1]" 2 --format latex
fk simple "[1,1,1]" 2 --format mathematica
```

The `--format` flag automatically enables symbolic printing and selects one of `pretty`, `inline`, `latex`, or `mathematica`.

The option `--weight` enables an additional “stratified” bound used by the boundedness tests in Phases 1–2. Unless one is explicitly computing such stratified truncations, this option should be left unset.

Reformatting saved output. When results are saved to disk (see below), the subcommand

```
fk print-as result.json --format latex
```

reformats a previously computed JSON result into a symbolic expression without rerunning the three-phase pipeline.

Config mode (reproducible runs and batches).

For reproducibility and large computations we recommend using configuration files. A template may be created by

```
fk template create my_run.yaml
fk template create my_run.yaml --overwrite
```

The resulting YAML file contains commented documentation for the supported keys. After editing, run

```
fk config my_run.yaml
```

Configuration files can be either JSON (always available) or YAML (requires PyYAML, installed via `pip install "fkcompute[yaml]"`).

The config loader supports two shapes:

- *single computation*: a file with top-level keys **braid** and **degree**;
- *batch mode*: a file with a top-level **computations** list, each entry containing a **name**, **braid**, and **degree**.

In batch mode, any additional top-level keys are treated as defaults and are merged into each computation unless overridden locally.

5.3 Output format

All entry points return the same JSON-serializable dictionary. For a single computation, the top-level structure is

```
{
  "terms": [...],
  "metadata": {...}
}
```

The list **terms** encodes a sparse polynomial in the topological variables and q . Each element has an x -exponent vector and a sparse q -polynomial coefficient:

```
{
  "x": [a1, a2, ...],
  "q_terms": [
    {"q": k1, "c": "n1"},
    {"q": k2, "c": "n2"},
    ...
  ]
}
```

Here **c** is stored as a decimal string to preserve arbitrary-precision integers produced by the FLINT backend. The **metadata** field records auxiliary information such as the braid word actually used (cyclic shifts may be applied during inversion), the inversion datum, the number of link components, and fixed fractional power offsets in the x - and q -exponents. When symbolic output is requested, these offsets are reintroduced automatically.

In batch mode (a config file with a top-level **computations** list), the return value is a dictionary mapping computation names to per-computation result dictionaries of the above form.

5.4 Python API

Programmatic access is provided by the single function `fkcompute.fk`. In “simple” mode, it is called with a braid list and a degree:

```
from fkcompute import fk

result = fk([1, 1, 1], degree=2)
```

The returned dictionary has the same **terms/metadata** structure as the CLI output.

The Python entry point additionally accepts keyword arguments controlling the pipeline. The most commonly used are:

- **threads**: number of threads used by the Phase 3 C++ backend;
- **max_workers** and **chunk_size**: parallelism settings for the Phase 1 inversion search;
- **include_flip** and **max_shifts**: control which braid variants are tried during inversion (cyclic shifts are always considered; a horizontal mirror is included only when **include_flip=true**);
- **save_data**, **save_dir**, **name**: enable saving of the inversion JSON, ILP CSV, and final result JSON;
- **symbolic**: attach a SymPy pretty-printed expression as **metadata["symbolic"]** (requires SymPy).

Two additional mechanisms are provided for resuming or accelerating computations:

- passing **inversion** or **inversion_file** skips Phase 1 by supplying a sign assignment directly;
- passing **ilp_file** skips Phases 1–2 and runs Phase 3 on a pre-generated ILP export.

These options are particularly useful when computing the same braid at several degrees: Phase 1 can be cached once and reused.

An advanced option **partial_signs** can be used to constrain the inversion search by fixing some strand signs while leaving others free. In the current implementation the “free” entries must be given as `None`

(Python) or `null` (YAML), while fixed entries are `+1` or `-1`.

Finally, `fk` also accepts a configuration path as its first argument. In that case it dispatches to the same config loader used by `fk config`.

5.5 Mathematica interface

The directory `mathematica/FkCompute/` contains a Wolfram Language packet which calls the Python implementation through a JSON bridge `python -m fkcompute.mathematica_bridge`. After installing the packet, one loads it in Mathematica via

```
PacletInstall[".../mathematica/FkCompute"]
Needs["FkCompute"]
```

and computes (for example) the trefoil by

```
FkCompute[{1, 1, 1}, 2]
```

When SymPy is available in the Python environment, the default return value is a Mathematica expression for the truncated polynomial in the variables corresponding to the link components and q .

The wrapper also exposes `FkComputeVersion[]` to query the Python and `fkcompute` versions seen by Mathematica. If Mathematica does not find the correct Python executable (for instance when using a virtual environment), set the executable explicitly, e.g.

```
SetFkComputePythonExecutable[
  "/path/to/python3"
]
```

The function `FkCompute` accepts options mirroring the Python keyword arguments (e.g. `"Threads"`, `"MaxWorkers"`, `"Verbose"`, `"SaveData"`, and `"Preset"`), which are passed through to the bridge unchanged.

5.6 Worked examples

We conclude with three representative examples.

Trefoil knot. The trefoil is the closure of σ_1^3 , encoded by `[1,1,1]`. In simple CLI mode:

```
fk simple "[1,1,1]" 2 --symbolic
```

This example is homogeneous, so Phase 1 is instantaneous.

Figure-eight knot. The figure-eight knot is represented by the 3-braid $\sigma_1\sigma_2^{-1}\sigma_1\sigma_2^{-1}$, encoded by `[1,-2,1,-2]`. In this case Phase 1 performs a search for an acceptable inversion datum. A typical workflow is to use the wizard (to tune workers/threads) or to store a reproducible configuration:

```
# figure_eight.yaml
braid: [1, -2, 1, -2]
degree: 5
max_workers: 8
threads: 8
save_data: true
name: figure_eight_d5
```

and run `fk config figure_eight.yaml`. The saved `figure_eight_d5_inversion.json` can then be reused at higher degrees.

A two-component link (Hopf link). The Hopf link is the closure of σ_1^2 , encoded by `[1,1]`. The computation returns a polynomial in two topological variables and q . For example:

```
fk simple "[1,1]" 2 --format latex
```

prints a LaTeX expression in variables x and y (one per component), while the JSON output encodes the same data via x -degree vectors of length two.

6 Performance

7 Validation and dataset

The F_K series is not defined by a simple closed formula: for non-homogeneous braids one must find an acceptable inversion datum and then evaluate a large

integer-point sum whose truncation depends on subtle boundedness properties. For this reason we validated the implementation using three independent checks, each targeting a different failure mode of the three-phase pipeline from Section 4.

7.1 Regression against reference tables

The first check is a direct coefficientwise comparison against a fixed table of precomputed reference values. In the repository this is implemented as a regression test suite `tests/test_fk_baseline.py`.

In addition, for development against externally published tables, the companion script `tests/compare_fks.py` compares locally computed JSON outputs against a downloaded snapshot of the topology.fyi FK table (see Section 7.4).

Inputs. The test enumerates a collection of knot configurations stored as YAML files in `tests/configs/config-10/`. Each configuration provides a knot identifier (e.g. `10_100`), a braid word (in the convention of Section 5.1), and (optionally) an inversion datum. The test restricts to knots with crossing number at most 11 and then runs `fkcompute.fk` at a fixed truncation degree (`MAX_DEGREE=10` in the test).

Reference data. For every configuration included in the parameter list, the repository contains a corresponding JSON output file in `tests/data_baseline/`. These JSON files use exactly the same output schema described in Section 5.3 (a sparse list of `terms` with x -degree vectors and sparse q -polynomial coefficients, together with meta-data such as the braid and the inversion). The baseline directory also contains the intermediate inversion JSON and ILP CSV exports for many examples (files ending in `_inversion.json` and `_ilp.csv`), which are useful for diagnosing mismatches by isolating the pipeline stage at which a discrepancy is introduced.

Comparison procedure. The test converts both the newly computed JSON result and the

baseline JSON file to explicit SymPy expressions in x and q (helper `data_file_to_sympy` in `tests/test_fk_baseline.py`). Since a change in truncation conventions could lead to one side containing fewer x -degrees than the other, the comparison is performed after truncating both expressions to the largest x -degree guaranteed to be present on both sides: if d_{new} and d_{base} are the maximal x -degrees in the two expressions, the test sets $d = \min(d_{\text{new}}, d_{\text{base}})$ and checks equality of the truncations to degree d . The resulting assertion is therefore an exact coefficientwise equality check within a common computed range.

This regression suite is primarily intended to guard against implementation regressions in Phases 2–3 (constraint reduction, export, enumeration, and polynomial arithmetic). When an inversion datum is supplied explicitly, those stages are fully deterministic; when Phase 1 is run as part of the computation, the same comparison still serves as an end-to-end check, and the saved inversion and ILP artifacts can be used to localize the source of any discrepancy.

7.2 The $q \rightarrow 1$ (Alexander/MMR) specialization

The second check uses the $q \rightarrow 1$ specialization predicted by the Melvin–Morton–Rozansky expansion, as recalled in Section 3.1. In the knot case this is the identity (5), equivalently $\lim_{q \rightarrow 1} F_K(x, q) = \text{s.e.}((x^{1/2} - x^{-1/2})/\Delta_K(x))$.

In the codebase this check is implemented for knots in `tests/test_bar_natan.py`. The test reads a collection of precomputed FK outputs from `tests/data/` (JSON files produced by running `fkcompute` with `save_data=true`). It then computes the Alexander-polynomial prediction from an independent planar diagram computation and compares the resulting x -series coefficients to those obtained by specializing the FK output at $q = 1$.

Extracting the $q = 1$ specialization from FK JSON. For a single-component result, the JSON output is a sparse sum $F_K(x, q) = \sum_d a_d(q) x^d$. The specialization at $q = 1$ is obtained termwise by eval-

uating each coefficient polynomial at 1: $a_d(1) = \sum_k c_{d,k}$, where $a_d(q) = \sum_k c_{d,k} q^k$. The helper `_fk_coeffs_q1` in `tests/test_bar_natan.py` computes the resulting integer coefficients $a_d(1)$ for all $0 \leq d \leq x_{\max}$, where $x_{\max}$ is read from `metadata["max_x_degrees"] [0]`.

Computing the Alexander prediction (independent of the FK pipeline). To avoid relying on any part of the FK implementation, the test computes the Alexander side using a separate SymPy translation of Bar–Natan’s Z -function algorithm. The function `bar_natan_Z` in `tests/bar_natan.py` takes a knot presented in KnotTheory’s PD notation $X[a, b, c, d]$ and returns a pair of rational functions $(P_0(T), P_1(T))$ in a variable T , where P_0 is (up to the usual $\pm T^k$ ambiguity) the Alexander polynomial. The algorithm follows the matrix-based formula of Bar–Natan and collaborators and is independent of the braid-based R -matrix evaluation used by `fkcompute`.

The test obtains a PD code for each knot in one of two ways. When available, it uses the PD notation stored in `tests/knotinfo.csv`. If the knot identifier is not present in that table, it constructs a PD code directly from the braid word stored in the FK JSON metadata (helpers `braid_to_pd` and `normalize_pd` in `tests/test_bar_natan.py`).

Finally, the predicted $q \rightarrow 1$ series is extracted from P_0 by expanding $(1 - 1/T)/P_0(T)$ at $T = 0$ to order $x_{\max}$ and comparing its coefficients to the list returned by `_fk_coeffs_q1`. Since Δ_K is only defined up to an overall sign, the test normalizes P_0 by requiring the leading coefficient of the numerator (after clearing denominators) to be positive.

7.3 The first q -derivative at $q = 1$ (first MMR correction)

The third check refines the $q \rightarrow 1$ specialization by testing the first q -derivative predicted by the next term in the MMR expansion. In the notation of Section 3.1, this is the constraint (7) on $D_q f_K(x, q)|_{q=1}$, which translates into a coefficientwise identity for $\partial_q F_K(x, q)|_{q=1}$ after the fixed normalization factor

$$(x^{1/2} - x^{-1/2}).$$

This check is carried out in the same test `tests/test_bar_natan.py`. Given FK output data in JSON form, the derivative at $q = 1$ is extracted directly from the sparse coefficient representation: $\partial_q a_d(q)|_{q=1} = \sum_k k c_{d,k}$ when $a_d(q) = \sum_k c_{d,k} q^k$. The helper `_fk_deriv_coeffs_q1` computes these integers for $0 \leq d \leq x_{\max}$. On the prediction side, the same Bar–Natan algorithm returns the first-order term $P_1(T)$, and the expected x -series is obtained by expanding $-(1 - 1/T) P_1(T)/P_0(T)^3$ at $T = 0$ to order $x_{\max}$. The test compares the coefficient lists exactly. Unlike P_0 , the expression governing P_1 is sign-invariant because it is computed from Δ_K^2 ; accordingly the sign normalization applied to P_0 does not affect the P_1 check.

7.4 Dataset availability

In addition to the above validation checks, we generated a dataset of computed F_K (and F_L) truncations which is distributed online at `topology.fyi`. Each dataset entry records the input braid word, the inversion datum used, the requested truncation degree(s), and the resulting sparse polynomial data in the JSON format described in Section 5.3.

The repository contains the scripts and input tables used to run such batch computations reproducibly. In particular, `tests/compute_fks.py` is a small runner which reads a YAML table of braids and precomputed inversion data (e.g. `tests/all_knots.yaml` and `tests/all_links.yaml`), invokes `fkcompute.fk` with `save_data=true` and the provided inversion to bypass Phase 1, and records per-instance timing and memory usage. The companion script `tests/compare_fks.py` implements an external consistency check against a downloaded snapshot of the `topology.fyi` FK table by parsing the website’s polynomial strings into SymPy and comparing them to locally computed JSON results up to a common x -degree.

Finally, while the three checks above validate the invariant values at the level of the final truncated series, the repository also contains targeted unit tests for the Phase 1 sign-diagram machin-

ery (arc labelling, permutation-based candidate generation, and deduplication). For example, `tests/test_sign_diagram_candidates.py` matches the exact candidate sets listed in `ImproveSignDiagramSearch-Davide.nb`, and `tests/test_perm_signs_knots.py` and `tests/test_perm_signs_links.py` verify that the inversion data used in batch runs (`tests/all_knots.yaml` and `tests/all_links.yaml`) is indeed produced by the permutation enumeration used by `fkcompute`.

8 Limitations and future work